

Quy Trình Nhận Dạng

Quy Trình Nhận Dạng	1
Cắt NIC ra khỏi nền	2
Phân loại ảnh	2
Xác định các vùng thông tin cần nhận dạng	3
Nhận dạng kí tự	3
Đánh Nhãn	3
Công cụ đánh nhãn	3
Các thông tin cần đánh nhãn	3
Dữ liệu đã được đánh nhãn	3
Huấn Luyện	4
Performance	4
Cấu trúc của mã nguồn huấn luyện	4
Serving	5
Mã nguồn dự đoán	5
Dev API	5
Production API	6
API	6
Deploy	6

Quy trình số hóa chứng minh thư trải qua các bước cơ bản:

1. Cắt đối tượng cần nhận dạng ra khỏi nền
2. Phân loại ảnh
3. Xác định các vùng thông tin cần nhận dạng
4. Nhận dạng kí tự



Cắt NIC ra khỏi nền

Cắt đối tượng ra khỏi nền hiệu quả nhất là sử dụng một mô hình segmentation thay vì sử dụng mô hình object detection để nhận dạng 4 góc.

Sau khi segment được NIC ra khỏi nền, chúng ta xác định 4 góc của NIC bằng thuật toán Douglas & Peucker Line Simplification và sử dụng phép biến đổi perspective transformation để đưa về ảnh chuẩn.

Phân loại ảnh

Sau khi cắt NIC ra khỏi nền, chúng ta sử dụng mô hình CNN để phân loại ảnh thành real/fake/photocopy/junk

Xác định các vùng thông tin cần nhận dạng

Để xác định thông tin cần nhận dạng thì hiệu quả nhất là sử dụng mô hình Faster-RCNN để nhận dạng các đối tượng trong bức ảnh, các đối tượng này là các thông tin cần nhận dạng như id, name, địa chỉ.

Trong mô hình Faster-RCNN, chúng ta cần điều chỉnh tham số anchor-size để phù hợp với bài toán nhận diện đối tượng là dòng text do các tham số trong mặc định trong mô hình thường được sử dụng để nhận diện các đối tượng trong đời sống thực tế.

Nhận dạng kí tự

Sau khi nhận dạng được các vùng thông tin ở trên, chúng ta sử dụng một mô hình OCR để các kí tự trong vùng ảnh đó.

Đánh Nhãn

Dữ liệu được đánh nhãn là phần quan trọng nhất để bảo mô hình chạy chính xác. Số lượng tối thiểu trong bài toán này là từ 10k ảnh trở lên cho mỗi loại ảnh.

Công cụ đánh nhãn

Sử dụng web tool [VIA](#) để đánh nhãn cho tất cả các bước

Các thông tin cần đánh nhãn

Chúng ta cần 2 lần đánh nhãn

1. Đánh segmentation mask của NIC bằng cách sử dụng 4 điểm, đồng thời phân loại NIC tại bước này
2. Đánh layout và gõ text cho NIC đã được tách ra khỏi nền.

Dữ liệu đã được đánh nhãn

Toàn bộ dữ liệu huấn luyện được lưu tại đây

Pwd: 123123

sftp://vmg@183.91.11.38/upload/nic/du_lieu_ban_giao/nic

Cấu trúc của thư mục dữ liệu

```
.
├── crop: dữ liệu đã xử lý để train cropper
│   ├── cropped
│   └── img
└── data: dữ liệu raw các toàn bộ dự án
```

```

| ┌ crop: dữ liệu phần crop
| ┌ doc: dữ liệu phân loại ảnh thành real/fake/junk/photo
| ┌ layout: dữ liệu phần layout detection
| ┌ ocr: dữ liệu raw phần ocr
|   ┌ ocr_textline: dữ liệu đã xử lý phần ocr
└ layout: dữ liệu đã xử lý để train layout detector
  ┌ img
└ ocr: dữ liệu đã xử lý để train OCR
  ┌ img

```

Huấn Luyện

Để huấn luyện tại các bước, chúng ta đều sử dụng các thư viện phổ biến. Cách sử dụng các thư viện này để huấn luyện và dự đoán đều được cung cấp

Giai đoạn	Thư viện
Tách đối tượng và xác định vùng thông tin cần nhận dạng	detectron2
OCR	vietocr

Source code quá trình huấn luyện đính kèm, bao gồm các giải thích để huấn luyện mô hình.

Performance

Kết quả độ chính xác của quá trình huấn luyện các mô hình

Giai đoạn	Performance
Crop	AP = 0.97
Layout	AP = 0.76
OCR	Precision = 0.8

Cấu trúc của mã nguồn huấn luyện

Để huấn luyện các mô hình, chúng ta sử dụng 2 thư viện detectron2 và vietocr. Các thư viện này đều đơn giản để sử dụng.

Mã nguồn được đính kèm có cấu trúc như sau,

```

.
├── crop
│   └── train.ipynb: huấn luyện mô hình cropper
├── layout
│   └── train.ipynb: huấn luyện mô hình layout detector
├── ocr
│   └── vietocr_gettingstart.ipynb: huấn luyện mô hình ocr

```

Serving

Các mô hình được huấn luyện ở bước trên và lưu lại toàn bộ tham số, trọng số. Lúc dự đoán chúng ta chỉ đơn giản là load trọng số và các tham số lên và chạy mô hình để dự đoán. Các phần dư thừa để huấn luyện sẽ không được thêm vào

Mã nguồn dự đoán

Toàn bộ mã nguồn dự đoán xem tại [đây](#).

Cấu trúc của mã nguồn

```

├── api.py: flask api
├── config: thư mục chứa các tham số
│   ├── config_dev.yml: tham số cho dev
│   ├── config.yml: tham số cho production
│   ├── dtree
│   │   └── config.yml
│   ├── gar.json
│   └── gunicorn_config.py: tham số cho api
├── Dockerfile: build docker image
├── fapi.py
├── pipeline.py: luồng xử lý của một ảnh
├── README.md
├── requirements.txt: toàn bộ thư viện cần thiết cho dự đoán
├── translate
│   └── translator.py

```

Dev API

Biến môi trường

CONFIG=/path/to/config.yml

Trước khi chạy api trên môi trường dev cần thay đổi file trọng số và file tham số tồn tại trên hệ thống đồng thời set biến môi trường

Production API

Khi chạy trên production, image docker sẽ tự tải trọng số và tham số trên cloud về, chạy api tự động

API

- Input: Một ảnh
- Output: json, chỉ trả ra các thông tin được nhận dạng thành công trong bức ảnh. Ví dụ, nếu mô hình thông nhận dạng được id thì sẽ không có trường id trong json ra về

Mặt trước	Mặt Sau
<pre>[{ "confidence": { "card_type": 0.9993, "id": 0.99967, "name": 0.999377, "side": 0.999694 }, "value": { "card_type": "real", "id": "၉/၀၀၁ (နိုင်)၀၂၂၁၃၅", "name": "မောင်ဝင်းတင့်", "side": "front" } }]</pre>	<pre>[{ "confidence": { "address": 0.999925, "card_type": 0.7111477, "side": 0.9985644 }, "value": { "address": "ဖိတ်စွတ်ကျေးရွာ တွဲတေးမြို့နယ်", "card_type": "real", "side": "back" } }]</pre>

Ví dụ:

```
curl -F 'file=@img.jpg' host:port/nic_img
```

Deploy

Toàn bộ source code được đóng gói dưới dạng docker. Tham khảo cách cài đặt nvidia-docker tại [đây](#).

Clone source code:

```
git clone https://github.com/pbcquoc/hades.git
```

Build Docker từ thư mục code:

```
docker build -t hades:v0 .
```

Để bật api:

```
docker run --gpus '"device=0"' -itd -p 30004:8000 hades:v0 -w {num_worker}
```

Trong đó:

- {num_worker} là số process để xử lý request
- 30004: là port để public ra mạng ngoài
- device=0: là gpu_id bắt đầu từ 0